

# 12a — GOF Design Patterns

## Key Concepts

- **GOF** = Gang of Four — 4 authors who catalogued 23 reusable design patterns in 1994
- Patterns are **blueprints**, not copy-paste code — proven solutions to recurring problems
- Three categories based on what problem they solve

| Category   | Question it answers                |
|------------|------------------------------------|
| Creational | How do I create objects?           |
| Structural | How do I compose objects together? |
| Behavioral | How do objects talk to each other? |

## Key Patterns

### Creational

| Pattern        | One-liner                                  | Key parts                                               |
|----------------|--------------------------------------------|---------------------------------------------------------|
| Singleton      | One globally shared instance               | Private constructor + static <code>getInstance()</code> |
| Factory Method | Create objects without knowing exact class | Factory decides which class to instantiate              |
| Builder        | Build complex objects step by step         | Chain method calls, end with <code>.build()</code>      |

### Structural

| Pattern   | One-liner                                  | Key parts                                               |
|-----------|--------------------------------------------|---------------------------------------------------------|
| Adapter   | Make incompatible interfaces work together | Wrapper translates one interface to another             |
| Decorator | Add behavior by wrapping objects           | Stack wrappers, each adds a layer                       |
| Proxy     | Control access to an object                | Sits in front of real object (auth, caching, lazy load) |
| Facade    | Simplify a complex subsystem               | One simple API hides internal complexity                |

### Behavioral

| Pattern  | One-liner                    | Key parts                                                        |
|----------|------------------------------|------------------------------------------------------------------|
| Observer | Notify many when one changes | Subject + list of Observers, Subject calls <code>Notify()</code> |
| Strategy | Swap algorithms at runtime   | Strategy interface + Concrete Strategies + Context               |

| Pattern        | One-liner                      | Key parts                                                           |
|----------------|--------------------------------|---------------------------------------------------------------------|
| <b>Command</b> | Encapsulate actions as objects | Command interface with <code>Execute()</code> + <code>Undo()</code> |

## How It Works

### Observer

1. Subject maintains a list of registered Observers
2. Subject's state changes
3. Subject calls `Notify()` — iterates through all Observers
4. Each Observer reacts independently

### Strategy

1. Define `IStrategy` interface with common method (e.g. `Pay()`)
2. Each algorithm is a Concrete Strategy class implementing the interface
3. Context holds a reference to `IStrategy`
4. Inject different strategy at runtime — no if/else needed

### Command

1. Define `ICommand` interface with `Execute()` and `Undo()`
2. Each action is a Concrete Command class
3. Invoker holds and triggers commands — doesn't know internals
4. Commands can be stored in a list, queued, replayed, or reversed

## Industry Examples

| Pattern   | Company           | Use Case                                                                          |
|-----------|-------------------|-----------------------------------------------------------------------------------|
| Singleton | Netflix           | Single config manager instance shared across services                             |
| Factory   | Stripe            | Creates right payment handler by currency/region                                  |
| Builder   | Microsoft         | <code>WebApplication.CreateBuilder()</code> in ASP.NET Core                       |
| Adapter   | Legacy migrations | REST-to-SOAP wrapper when modernizing old systems                                 |
| Decorator | ASP.NET Core      | Middleware pipeline — auth, logging, compression stacked in layers                |
| Proxy     | Netflix           | Service-to-service proxy handles retries, circuit breaking, logging               |
| Facade    | Stripe            | <code>stripe.createCharge()</code> hides fraud checks, bank calls, ledger updates |
| Observer  | YouTube           | Notifications — one event, many subscribers react independently                   |
| Strategy  | Uber              | Surge pricing — different pricing algorithms swapped by region/time               |
| Command   | MediatR (.NET)    | Every request is a Command dispatched to a handler                                |

| Pattern | Company | Use Case                                                          |
|---------|---------|-------------------------------------------------------------------|
| Command | AWS SQS | Queue jobs are Command objects serialized and executed by workers |
| Command | Git     | Every commit is a Command — stored, replayable, reversible        |

## Common Interview Traps

| Confusion           | Clarification                                                                    |
|---------------------|----------------------------------------------------------------------------------|
| Decorator vs Proxy  | Decorator <i>adds behavior</i> ; Proxy <i>controls access</i>                    |
| Strategy vs Command | Strategy swaps <i>how</i> (algorithm); Command encapsulates <i>what</i> (action) |
| Observer vs Kafka   | Observer has no ordering/retry guarantees; Kafka adds those for scale            |
| Singleton risk      | Not thread-safe by default — needs locking in multi-threaded environments        |

## Interview Questions

### Q1: What are the three categories of GOF patterns?

Creational (object creation), Structural (object composition), Behavioral (object communication)

### Q2: What's the difference between Decorator and Proxy?

Both wrap an object — Decorator adds behavior, Proxy controls access (auth, caching, lazy load)

### Q3: Which GOF pattern does ASP.NET Core middleware implement?

Decorator — each middleware wraps the next, adding behavior in layers

### Q4: How is Strategy different from Command?

Strategy swaps how something is done (algorithm); Command encapsulates what is done (action) so it can be queued, logged, or undone

### Q5: Where does Observer break down at scale?

Uncontrolled cascades, memory leaks from unsubscribed observers, no ordering guarantees — production systems use Kafka instead

### Q6: Why use Builder over a constructor with many parameters?

Constructors with many params are error-prone and unreadable; Builder lets you set only what you need with named, chainable calls

### Q7: What problem does Factory solve?

Decouples object creation from usage — caller doesn't need to know which concrete class to instantiate, making it easy to swap implementations